

Лабораторная работа №14

Программирование в командном процессоре ОС UNIX. Расширенное программирование

Студент: Щеголяев Максим Ильич

Группа: НБИбд-02-25

Номер студенческого билета: 1032256490

Цель работы

Изучение расширенных возможностей shell-программирования в UNIX, освоение взаимодействия процессов, организации блокировок ресурсов, создания пользовательских утилит и генерации случайных последовательностей.

Введение

Командная оболочка Bash является одним из ключевых инструментов администрирования UNIX-систем. С помощью shell-скриптов можно автоматизировать выполнение системных задач, управлять процессами, работать с файлами и организовывать взаимодействие программ.

Shell-программирование применяется:

- в системном администрировании;
- DevOps и CI/CD;
- автоматизации серверов;
- резервном копировании;
- мониторинге инфраструктуры.

Теоретические сведения

Командный процессор Bash

Bash представляет собой интерпретатор командной строки, поддерживающий:

- переменные;
- функции;
- циклы;
- условные конструкции;
- перенаправление потоков ввода/вывода.

Shell-сценарии позволяют объединять стандартные UNIX-утилиты в полноценные программы.

Механизмы синхронизации процессов

При одновременном доступе нескольких процессов к одному ресурсу возникают конфликты. Для предотвращения ошибок используются механизмы синхронизации:

- семафоры;
- mutex;
- lock-файлы.

В лабораторной работе используется lock-файл как простой механизм блокировки.

Справочная система UNIX

Команда `man` предоставляет документацию по программам Linux. Файлы справки находятся в каталоге: `/usr/share/man/`

Справочная информация хранится в сжатом формате.

Генерация случайных чисел

Переменная `RANDOM` в Bash генерирует псевдослучайные числа в диапазоне от 0 до 32767.

Практическая часть

Задание 1. Реализация механизма семафоров

Постановка задачи

Требуется реализовать shell-скрипт, обеспечивающий блокировку ресурса при обращении нескольких процессов.

Код программы

```
#!/bin/bash

LOCKFILE="/tmp/resource.lock"

while true
do
    if [ -f "$LOCKFILE" ]; then
        echo "Ресурс занят другим процессом"
        sleep 2
    else
        touch "$LOCKFILE"
        echo "Ресурс захвачен"
        sleep 5
        rm -f "$LOCKFILE"
        echo "Ресурс освобожден"
        break
    fi
done
```

```
fi
done
```

Принцип работы

Сценарий проверяет наличие lock-файла. Если файл существует — ресурс считается занятым. Если файл отсутствует — процесс создаёт lock-файл и получает доступ к ресурсу.

Результаты выполнения

При запуске нескольких процессов одновременно только один процесс получает доступ к ресурсу. Остальные процессы переходят в режим ожидания.

Скриншоты

1. Запуск нескольких процессов
2. Ожидание освобождения ресурса
3. Успешное завершение работы

Задание 2. Реализация аналога команды man

Код программы

```
#!/bin/bash

FILE="/usr/share/man/man1/$1.1.gz"

if [ -f "$FILE" ]; then
    less "$FILE"
else
    echo "Справка для команды отсутствует"
fi
```

Анализ работы программы

Сценарий проверяет существование файла справки. При наличии файла выполняется его открытие через less.

Особенности реализации

- используется условная конструкция if;
- применяется проверка существования файла;
- реализована обработка ошибок.

Задание 3. Генерация случайной последовательности

Код программы

```
#!/bin/bash

for ((i=0; i<20; i++))
do
    CHAR=$((RANDOM % 26 + 65))
    printf "\$(printf '%03o' $CHAR)"
done

echo
```

Объяснение работы

Случайное число преобразуется в ASCII-код латинской буквы.

Результаты выполнения

Пример результата: QWERTYASDFZXCVBNM

Анализ результатов лабораторной работы

В ходе выполнения работы были изучены методы организации взаимодействия процессов в UNIX-системах.

Реализация семафоров показала принципы блокировки ресурсов и предотвращения конфликтов доступа.

Работа со справочной системой Linux продемонстрировала организацию документации UNIX.

Использование RANDOM позволило реализовать генератор случайных строк без сторонних библиотек.

Преимущества Bash

Преимущество	Описание
Простота	Быстрое создание сценариев
Автоматизация	Упрощение рутинных задач
Интеграция	Работа с системными утилитами
Скорость разработки	Не требуется компиляция

Недостатки Bash

Недостаток	Описание
Низкая производительность	Медленнее компилируемых языков
Ограниченность	Сложно создавать большие проекты
Ошибки синтаксиса	Bash чувствителен к пробелам

Ответы на контрольные вопросы

1. В конструкции while [\$1 != "exit"] отсутствуют пробелы около квадратных скобок.
2. Конкатенация строк выполняется простым объединением:

```
str="$a$b"
```

3. Команда seq генерирует последовательности чисел.
4. Выражение \$((10/3)) возвращает 3.
5. zsh отличается улучшенным автодополнением и расширенной настройкой оболочки.
6. Конструкция for ((a=1; a <= LIMIT; a++)) является корректной.
7. Bash удобен для автоматизации, но уступает Python и C++ при разработке сложных приложений.

Выводы

В результате выполнения лабораторной работы были:

- изучены расширенные возможности Bash;
- освоены механизмы синхронизации процессов;
- реализованы shell-сценарии;
- исследована работа справочной системы UNIX;
- получены навыки автоматизации задач.

Дополнительный раздел 1

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 2

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 3

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 4

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и

мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 5

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 6

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.

Дополнительный раздел 7

Shell-программирование активно применяется в администрировании Linux-серверов. Сценарии позволяют автоматизировать резервное копирование, управление службами, анализ журналов и мониторинг процессов.

В современных системах Bash используется совместно с:

- Docker;
- Kubernetes;
- GitLab CI/CD;
- Ansible;
- systemd.

Автоматизация через shell-скрипты снижает вероятность человеческих ошибок и повышает эффективность обслуживания инфраструктуры.